Radio Columbia — Documentación técnica y de handoff

Proyecto: sitio web de Radio Columbia (Costa Rica)
Stack: Next.js 14 (App Router) · React 18 · TypeScript/JavaScript · TailwindCSS 3.4
Hosting: Vercel · **Contenido:** WordPress autoalojado (REST API)

Tabla de contenido

1. Visión general
2. Arquitectura
3. Estructura del proyecto
4. Puesta en marcha local
5. Variables de entorno
6. Dependencias
7. Convenciones de código
8. Guía de Clean Code
9. Reglas invariantes del proyecto
10. Checklist de despliegue
11. Buenas prácticas de handoff
12. Operación y resolución de problemas
13. Glosario

---

1. Visión general

Radio Columbia es una aplicación web para una emisora costarricense. Ofrece:

- Transmisión de radio en vivo con reproductor persistente.
- Noticias publicadas en WordPress.
- Podcasts y radionovelas desde feeds RSS (Captivate y listas de YouTube).
- Parrilla de programación.
- Widgets: clima, sismos, resultados de lotería, tabla de posiciones deportivas y conversor de moneda.
- SEO completo: sitemap, robots, Open Graph, datos estructurados Schema.org.
- Analítica con Google Analytics.
- Formulario de contacto vía EmailJS.

2. Arquitectura

2.1 Flujo de datos de WordPress

El origen de WordPress es el punto más sensible en carga del sistema, por lo que **toda lectura se cachea**.

```
navegador ──► /api/wp/[...path] ──► Data Cache de Next.js ──► WordPress
                                              ▲
componentes de servidor ──────────────────────┘  (directo, mismas etiquetas)

Publicación en WordPress ──► /api/revalidate ──► revalidateTag() ──► caché purgado
```

Pieza	Archivo	Responsabilidad
Servicio WordPress	`src/lib/wordpressService.ts`	Único punto de acceso a WordPress. Todas las lecturas pasan por `wpFetch`: en servidor consulta WordPress con Data Cache etiquetado; en navegador llama a `/api/wp`. Usa `_fields` para reducir la carga.
Proxy de lectura	`src/app/api/wp/[...path]/route.ts`	Solo lectura. Lista blanca de colecciones y parámetros, limita `per_page`, reenvía `X-WP-Total` y `X-WP-TotalPages`, define políticas de caché por colección.
Webhook de purga	`src/app/api/revalidate/route.ts`	Recibe el aviso de WordPress al publicar y purga las etiquetas relevantes. Rechaza todo si no hay secreto configurado.
Origen WordPress	`src/lib/wpOrigin.ts`	Lee `WORDPRESS_API_URL` (sin prefijo `NEXT_PUBLIC_`, a propósito).

Etiquetas de caché (definidas solo en `WP_TAGS`, `wordpressService.ts`): `wp`, `wp-posts`, `wp-categories`, `wp-tags` y `wp-post-<slug>` para artículos individuales.

Ventana de respaldo: `FALLBACK_REVALIDATE = 600` s. Si un aviso del webhook se pierde, el contenido queda desactualizado como máximo 10 minutos.

La instalación del mu-plugin en WordPress está descrita en `docs/wordpress-revalidate.md`.

2.2 Otras fuentes de datos

Servicio RSS/Podcasts (`src/lib/rssService.ts`, `src/lib/rssConfig.ts`)

- Maneja listas de YouTube (convertidas a feeds Atom) y feeds RSS de Captivate.
- Singleton con caché en memoria (1 hora).
- Procesamiento por lotes (5 feeds a la vez) con reintentos y control de tasa.
- Sistema de proxy de respaldo para problemas de CORS.
- Dos categorías: `podcast` y `radionovela`.

Rutas de API (`src/app/api/*`)

Ruta	Función
`/api/wp/[...path]`	Proxy de lectura a WordPress
`/api/revalidate`	Webhook de purga de caché
`/api/rss`	Proxy de feeds RSS
`/api/earthquakes`	Datos sísmicos
`/api/jps/chances/last`	Último resultado de Chances
`/api/jps/loteria/last`	Último resultado de Lotería
`/api/standings`	Tabla de posiciones deportivas

Ruta	Función
`/api/weather`	Clima por ciudad

2.3 Contextos de React

- **PlayerContext** (`src/lib/PlayerContext.tsx`): estado global del reproductor. Dos modos: `radio` (en vivo) y `podcast` (episodios). Gestiona la navegación entre episodios.
- **SearchContext** (`src/lib/SearchContext.tsx`): estado global de búsqueda.

2.4 Páginas y layout

- App Router con grupos de rutas. Artículos en la raíz (`/[slug]`); podcasts en `/podcasts/[id]`.
- Inicio, `/news` y `/sports` son componentes de cliente. Su HTML base es un prerender estático servido desde el edge de Vercel; los datos de noticias se piden en cliente a través de `/api/wp`.
- `src/app/layout.js` envuelve la app con `SearchProvider` → `PlayerProvider`, incluye `SplashPage`, `RadioPlayer` y los datos estructurados de `RadioStation`.
- El reproductor debe persistir entre navegaciones (vive en el layout global).

2.5 Tipos

- `src/types/podcast.ts`: feeds RSS, shows y episodios.
- `src/types/wordpress.ts`: posts, categorías y etiquetas.

2.6 Estilos

TailwindCSS 3.4 (`tailwind.config.js`, fuente Outfit), alias `@/*` → `src/*`, estilos globales en `src/app/globals.css`.

3. Estructura del proyecto

```
radio-columbia/
├── docs/                       Documentación (esta guía, mu-plugin de WordPress)
├── public/                     Recursos estáticos (isotipo, manifest, assets)
├── src/
│   ├── app/
│   │   ├── (pages)/            about-us, contact, news, podcasts, sports, [slug]
│   │   ├── api/                Rutas de API (ver 2.2)
│   │   ├── layout.js           Layout global, SEO, analítica
│   │   ├── page.jsx            Inicio
│   │   ├── sitemap.js          Sitemap dinámico
│   │   ├── robots.js           Reglas para rastreadores
│   │   └── opengraph-image.js  Imagen Open Graph
│   ├── components/
│   │   ├── home/  news/  podcasts/  about-us/  contact/
│   │   ├── UI/                 NewsCard, NewsCardGrid, PodcastCard
│   │   ├── SEO/JsonLd.jsx      Datos estructurados
│   │   └── Navbar, Footer, RadioPlayer, SplashPage
│   ├── lib/                    Servicios, contextos, SEO, parrilla (schedule.ts)
│   └── types/                  Tipos TypeScript
├── next.config.mjs             Configuración efectiva de Next.js
├── tailwind.config.js
└── package.json
```

> **Nota sobre configuración:** el archivo vigente es `next.config.mjs` (imágenes remotas, cabeceras de seguridad, caché de sitemap/robots). Existen también `next.config.ts`, `postcss.config.js`/`.mjs` y `lib/api.js` en la raíz; si se limpian, conviene verificar primero que no se usan.

4. Puesta en marcha local

Requisitos: Node.js 20 o superior y npm.

```
npm install
# crear .env.local con las variables de la sección 5
npm run dev        # http://localhost:3000
```

Comando	Descripción
`npm run dev`	Servidor de desarrollo
`npm run build`	Build de producción
`npm run start`	Servidor de producción local
`npm run lint`	ESLint

Importante: nunca ejecutar `npm run build` con el servidor de desarrollo activo. Ambos escriben en `.next/` y generan grafos de chunks incompatibles, lo que produce el error engañoso `Cannot find module './vendor-chunks/*.js'`. Para recuperarse, borrar la salida compilada conservando las cachés:

```
find .next -mindepth 1 -maxdepth 1 ! -name cache -exec rm -rf {} +
```

5. Variables de entorno

Solo se documentan **nombres**; los valores viven en `.env.local` (ignorado por git) y en Vercel.

Variable	Requerida	Propósito
`WORDPRESS_API_URL`	**Sí** (el build falla sin ella)	Base REST de WordPress, sin barra final
`WP_REVALIDATE_SECRET`	No (degrada con gracia)	Secreto compartido para `/api/revalidate`
`WEATHER_API_KEY`	Para el widget de clima	Clave del proveedor de clima
`NEXT_PUBLIC_SITE_URL`	No (tiene valor por defecto)	URL pública del sitio para SEO, sitemap y robots
`NEXT_PUBLIC_GA_ID`	No (tiene valor por defecto)	ID de medición de Google Analytics
`NEXT_PUBLIC_RADIO_STREAM_URL`	No (tiene valor por defecto)	URL del stream de radio en vivo
`NEXT_PUBLIC_EMAILJS_SERVICE_ID`	Para el formulario de contacto	Servicio de EmailJS
`NEXT_PUBLIC_EMAILJS_TEMPLATE_ID`	Para el formulario de contacto	Plantilla de EmailJS
`NEXT_PUBLIC_EMAILJS_PUBLIC_KEY`	Para el formulario de contacto	Clave pública de EmailJS

Reglas:

- `WORDPRESS_API_URL` **no** lleva prefijo `NEXT_PUBLIC_`: así no se incluye en el bundle del navegador. No añadirlo (ver `src/lib/wpOrigin.ts`).
- Toda variable `NEXT_PUBLIC_*` es visible para cualquier visitante; nunca poner secretos en ellas.
- `.gitignore` tiene una regla `.env*`. Si se crea o cambia un `.env.example`, hay que añadirlo con `git add -f`.
- Tras cambiar variables en Vercel, hay que **redesplegar** para que el runtime las vea.

6. Dependencias

6.1 Producción

Paquete	Versión	Uso
`next`	^14.2.33	Framework (App Router)
`react`, `react-dom`	^18.2.0	UI
`@next/third-parties`	^16.3.0	Integración de Google Analytics
`framer-motion`	^12.23.26	Animaciones
`lucide-react`	^0.525.0	Iconos
`react-icons`	^5.0.1	Iconos adicionales
`react-slick`, `slick-carousel`	^0.30.3, ^1.8.1	Carruseles
`@emailjs/browser`	^4.4.1	Envío del formulario de contacto
`xml2js`	^0.6.2	Parseo de feeds RSS/Atom
`@types/xml2js`	^0.4.14	Tipos de `xml2js`

6.2 Desarrollo

Paquete	Versión	Uso
`typescript`	^5.3.3	Tipado
`tailwindcss`	^3.4.1	Estilos
`postcss`, `autoprefixer`	^8.4.31, ^10.4.21	Procesamiento CSS
`@types/node`, `@types/react`, `@types/react-dom`	^20.11.0, ^18.2.0, ^18.2.0	Tipos

6.3 Servicios externos

Servicio	Uso
WordPress (REST API)	Noticias, categorías, etiquetas
Captivate.fm	Feeds y artwork de podcasts
YouTube	Listas de reproducción convertidas a Atom
radio.co	Stream de radio en vivo
EmailJS	Correo del formulario de contacto
Google Analytics	Métricas
Vercel	Hosting, Data Cache, edge

6.4 Política de actualización

- Actualizar dependencias en una rama aparte, con `npm install` y `npm run build` limpios antes de fusionar.
- Subidas de versión mayor de `next` o `react` requieren revisar las reglas invariantes (sección 9), especialmente el comportamiento del Data Cache.
- Mantener `package-lock.json` versionado; no borrarlo para "arreglar" instalaciones.
- Ejecutar `npm audit` periódicamente y revisar los avisos antes de actualizar a ciegas.

7. Convenciones de código

7.1 Generales

- **Idioma:** comentarios y documentación en español; identificadores de código en inglés.
- **Alias de importación:** usar `@/…` en lugar de rutas relativas largas (`../../..`).
- **TypeScript para código nuevo.** Hay archivos `.js/.jsx` heredados; se migran de forma oportunista, no en bloque.
- **Tipado estricto:** `strict: true`. Evitar `any`; si es inevitable, comentar el motivo.
- **Componentes cliente:** añadir `'use client'` solo donde se necesiten hooks, eventos o APIs del navegador. Por defecto, componentes de servidor.

7.2 Nombres

Elemento	Convención	Ejemplo
Componentes React	PascalCase, un componente por archivo	`NewsCard.tsx`
Funciones y variables	camelCase	`getPostBySlug`
Constantes de módulo	UPPER_SNAKE_CASE	`FALLBACK_REVALIDATE`
Tipos e interfaces	PascalCase	`ScheduleBlock`
Clases de error	Sufijo `Error`	`WordPressUnavailableError`
Carpetas de rutas	kebab-case	`about-us`

7.3 Organización

- Acceso a datos externos solo en `src/lib/*` o en `src/app/api/*`, nunca dentro de componentes de presentación.
- Componentes agrupados por dominio (`home`, `news`, `podcasts`…); los reutilizables en `components/UI`.
- Tipos compartidos en `src/types`.
- Datos de configuración (feeds, parrilla) separados de la lógica: `rssConfig.ts`, `schedule.ts`.

7.4 Estilos

- Tailwind por utilidades; evitar CSS a medida salvo en `globals.css`.
- Diseño *mobile first*: probar siempre en ancho móvil además de escritorio.
- Imágenes con `next/image`; los dominios remotos permitidos se declaran en `next.config.mjs`.

7.5 Comentarios

Los comentarios explican **por qué**, no **qué**. El código existente documenta decisiones de carga y caché; mantener ese nivel y estilo al tocar esas zonas.

7.6 Git

- Una rama por tarea; commits pequeños y descriptivos, en español, en imperativo o descripción breve del cambio (por ejemplo, "Calendario de programas").
- No se suben secretos, `.env.local` ni `.next/`.
- Revisar `git status` y `git diff` antes de cada commit.
- Los cambios se integran a `main` mediante Pull Request.

8. Guía de Clean Code

Principios aplicados a este proyecto.

8.1 Nombres que revelan intención

```
// Mal
const d = await f(s);

// Bien
const post = await getPostBySlug(slug);
```

Evitar abreviaturas crípticas; el nombre debe responder qué es y para qué sirve.

8.2 Funciones pequeñas con una sola responsabilidad

Una función hace una cosa y está en un solo nivel de abstracción. Si necesita un comentario para separar "bloques", son funciones distintas. Ejemplo: `authorized()` en `/api/revalidate` solo valida el secreto; el `POST` solo orquesta.

8.3 Evitar la duplicación (DRY)

- Las etiquetas de caché viven en un único objeto (`WP_TAGS`). No escribir strings de etiquetas sueltos.
- Si la misma lógica aparece tres veces, extraerla a `src/lib` o a un componente en `UI`.
- No extraer antes de tiempo: dos usos similares no siempre justifican una abstracción.

8.4 Manejo de errores explícito

- Fallar de forma clara y tipada (`WordPressUnavailableError`), no devolver `null` ambiguo.
- No tragar excepciones con `catch {}` vacíos.
- Distinguir "no existe" (404 real) de "no se pudo consultar" (error de transporte). Ver sección 9.

8.5 Sin números ni textos mágicos

```
// Mal
next: { revalidate: 600 }

// Bien
const FALLBACK_REVALIDATE = 600;
next: { revalidate: FALLBACK_REVALIDATE }
```

8.6 Componentes de UI

- Componentes de presentación puros: reciben props, no hacen *fetch*.
- Estado lo más cerca posible de donde se usa; contexto global solo para lo realmente global (reproductor, búsqueda).
- Efectos (`useEffect`) con limpieza y dependencias correctas; no usarlos para derivar datos que se pueden calcular en el render.
- Listas con `key` estable (id o slug), nunca el índice si el orden cambia.

8.7 Código muerto y comentarios obsoletos

Borrar código comentado, imports sin uso y archivos huérfanos. El historial de git conserva lo que se elimine.

8.8 Regla del boy scout

Dejar el código un poco mejor de como se encontró, sin mezclar refactors grandes con cambios funcionales en el mismo commit.

8.9 Lista de verificación antes de abrir un PR

- [ ] Los nombres se entienden sin leer la implementación.

- [ ] Ninguna función hace más de una cosa.
- [ ] No hay duplicación evidente ni valores mágicos.
- [ ] Los errores se manejan y no se ocultan.
- [ ] No hay `console.log`, código comentado ni imports sin uso.
- [ ] No se rompe ninguna regla invariante (sección 9).
- [ ] `npm run lint` y `npm run build` pasan sin errores.

9. Reglas invariantes del proyecto

Romperlas reintroduce caídas que el sitio ya tuvo. **No son preferencias de estilo.**

1. **Nunca consultar WordPress directamente desde el navegador.** Toda lectura pasa por `WordPressService` para heredar el proxy, las etiquetas de caché y el recorte con `_fields`. Un `fetch` directo desde el cliente implica una petición al origen por visitante.
2. **Nunca reintentar una petición fallida a WordPress.** Reintentar contra un origen saturado multiplica la carga justo cuando no tiene capacidad. Un solo intento y un fallo limpio es deliberado.
3. **Nunca convertir un fallo de transporte en `notFound()`.** `getPostBySlug` lanza `WordPressUnavailableError` cuando WordPress no responde, y `src/app/(pages)/[slug]` lo deja propagarse a propósito. Un 404 se cachearía y sobreviviría a la caída, informando a los buscadores de que artículos reales no existen.
4. **Mantener `_fields` en toda consulta**, y nunca pedir `per_page=100` junto con `_embed`. El sitemap solo necesita `slug` y `modified` (ver `getPostSlugs`).
5. **No consultar por categoría dentro de un bucle.** `CategoryNewsGrid` pide todas las categorías en una llamada y agrupa en memoria.
6. Los feeds de podcast se configuran en `rssService.ts` (`PODCAST_RSS_FEEDS`); las listas de YouTube necesitan metadatos en `YOUTUBE_PODCAST_CONFIG`.
7. El servicio RSS tiene su propia lógica de reintentos y caché: no se debe saltar.
8. El reproductor debe permanecer persistente entre navegaciones (contexto global en el layout).

10. Checklist de despliegue

10.1 Antes del despliegue

- [ ] Rama actualizada con `main` y sin conflictos.
- [ ] `npm install` sin errores; `package-lock.json` actualizado y versionado.
- [ ] `npm run lint` sin errores.
- [ ] `npm run build` exitoso en local (con el servidor de desarrollo detenido).
- [ ] Revisión manual de las vistas clave en móvil y escritorio: inicio, noticias, artículo, podcasts, deportes, contacto.
- [ ] El reproductor reproduce la radio y sigue sonando al navegar.
- [ ] Sin secretos ni valores sensibles en el diff (`git diff`).
- [ ] Si se añadió una variable de entorno: creada en Vercel (Preview y Production) y documentada en la sección 5.
- [ ] Si se añadió un dominio de imágenes: declarado en `next.config.mjs`.
- [ ] Pull Request revisado y aprobado.

10.2 Configuración en Vercel

- [ ] `WORDPRESS_API_URL` definida (sin ella el build falla).
- [ ] `WP_REVALIDATE_SECRET` definida y **idéntica** a la del mu-plugin de WordPress.
- [ ] `WEATHER_API_KEY` y variables de EmailJS definidas.
- [ ] Variables `NEXT_PUBLIC_*` opcionales revisadas (sitio, analítica, stream).
- [ ] Redespliegue realizado tras cualquier cambio de variables.

10.3 Después del despliegue (verificación)

- [ ] Portada y `/news` cargan con contenido.
- [ ] Un artículo abre y muestra imagen, título y contenido.
- [ ] Webhook operativo:

  ```
  curl -s https://columbia.co.cr/api/revalidate
  # {"ok":true,"method":"POST","secretConfigured":true}
  ```

  Si `secretConfigured` es `false`, falta la variable o falta redesplegar.
- [ ] Publicar un post de prueba en WordPress y comprobar que aparece en la portada en pocos segundos.
- [ ] Widgets (clima, sismos, lotería, posiciones) responden.
- [ ] Podcasts y radionovelas cargan episodios.
- [ ] `/sitemap.xml` y `/robots.txt` responden correctamente.
- [ ] Formulario de contacto envía un mensaje de prueba.
- [ ] Google Analytics registra la visita en tiempo real.
- [ ] Revisar los logs de Vercel en busca de errores en los primeros minutos.

10.4 Plan de reversión

1. En Vercel → Deployments, promover el despliegue estable anterior (*Promote to Production* / *Instant Rollback*).
2. Confirmar la portada y un artículo.
3. Abrir una incidencia con la causa y corregir en una rama nueva.

11. Buenas prácticas de handoff

Qué debe quedar claro para quien reciba el proyecto.

11.1 Accesos y propiedad (entregar por canal seguro)

Lista de lo que la persona receptora debe tener, **sin escribir credenciales en documentos ni en git**:

- Repositorio GitHub (permisos adecuados).
- Proyecto de Vercel y sus variables de entorno.
- Panel de administración de WordPress y acceso al servidor/hosting donde está el mu-plugin.
- Cuentas de EmailJS, Google Analytics, proveedor de clima y Captivate.
- Dominio y DNS.

Los secretos se comparten mediante un gestor de contraseñas, nunca por correo o chat en texto plano.

11.2 Contexto que no se deduce del código

- **Por qué** existen las reglas invariantes (sección 9) y qué incidentes las motivaron.
- Qué partes son heredadas (archivos `.js/.jsx`, configuraciones duplicadas) y cuáles son el patrón a seguir.
- Quién es el contacto del lado de WordPress y de la emisora para contenido, parrilla y feeds.
- Qué hay pendiente: lista de tareas abiertas, deuda técnica conocida y decisiones aplazadas, en el gestor de incidencias del repositorio.

11.3 Procedimiento de entrega

1. **Sesión de traspaso (60–90 min):** recorrido por la arquitectura (sección 2), flujo de caché y despliegue.
2. **Ejecución guiada:** la persona receptora levanta el proyecto local desde cero con esta guía, y realiza un cambio pequeño de punta a punta (rama → PR → despliegue).
3. **Período de acompañamiento:** disponibilidad acordada para dudas durante las primeras semanas.
4. **Confirmación por escrito** de que accesos, documentación y pendientes fueron recibidos.

11.4 Mantener la documentación viva

- Cambios de arquitectura, variables de entorno o proceso de despliegue **se documentan en el mismo PR** que los introduce.
- Esta guía y `CLAUDE.md` deben mantenerse coherentes.
- No documentar valores secretos, límites de capacidad ni debilidades sin corregir en archivos versionados; eso va en notas operativas internas.

11.5 Checklist de handoff

- [ ] Accesos transferidos y verificados (sección 11.1).
- [ ] El proyecto levanta en local siguiendo solo esta documentación.
- [ ] Variables de entorno documentadas y existentes en Vercel.
- [ ] Último despliegue estable identificado.
- [ ] Pendientes y deuda técnica registrados en el gestor de incidencias.
- [ ] Contactos clave (WordPress, emisora, hosting) entregados.
- [ ] Sesión de traspaso realizada.
- [ ] Confirmación de recepción firmada o por escrito.

12. Operación y resolución de problemas

Síntoma	Causa probable	Qué hacer
El build falla con "Falta la variable de entorno WORDPRESS_API_URL"	Variable no definida	Definirla en `.env.local` o en Vercel y redesplegar
`Cannot find module './vendor-chunks/*.js'`	`build` y `dev` ejecutados a la vez	Borrar `.next` conservando `cache` (sección 4)
Contenido nuevo tarda ~10 minutos en aparecer	Webhook no llega o secreto distinto	Verificar mu-plugin, secreto y `secretConfigured` (sección 10.3)
`/api/revalidate` responde 401	El secreto no coincide	Igualar el valor en Vercel y en WordPress; redesplegar
Artículos devuelven error en vez de 404	WordPress no responde (comportamiento intencional)	Revisar el estado del servidor WordPress; no convertir en `notFound()`
Imágenes no cargan	Dominio no permitido	Añadirlo a `images.remotePatterns` en `next.config.mjs`
Widget de clima vacío	Falta `WEATHER_API_KEY`	Definir la variable y redesplegar
Formulario de contacto falla	Variables de EmailJS ausentes o incorrectas	Revisar las tres variables `NEXT_PUBLIC_EMAILJS_*`
Podcast no aparece	Feed sin configurar	Revisar `PODCAST_RSS_FEEDS` / `YOUTUBE_PODCAST_CONFIG`

13. Glosario

- **Data Cache:** caché de Next.js para respuestas de `fetch`, con etiquetas y revalidación.
- **Etiqueta de caché (tag):** nombre que permite purgar un grupo de entradas con `revalidateTag()`.
- **mu-plugin:** *must-use plugin* de WordPress; se activa solo al estar en `wp-content/mu-plugins/`.
- **Webhook:** aviso HTTP que WordPress envía a `/api/revalidate` cuando cambia un post.
- **`_fields`:** parámetro de la REST API de WordPress que limita los campos devueltos.
- **Proxy de lectura:** ruta `/api/wp` que intermedia entre el navegador y WordPress.
- **Handoff:** traspaso formal de un proyecto a otra persona o equipo.